

Data Structure and Programming, Spring 2025

Written Homework #5

Pei-Yuan, Wu peiyuanwu@ntu.edu.tw
Yi- Chou Lin b10901149@ntu.edu.tw
Chiao-Yun Chin b10901154@ntu.edu.tw
Yi-Chen Wang b10901143@ntu.edu.tw

RELEASE DATE: 05/15/2025
DUE DATE: 05/29/2025

Late policy: $\text{Score} = \text{Score} \times 0.7^{\lceil h \rceil / 24}$, $h = \text{late hours} \leq 48$

1. (16%) Binary Heap Operation

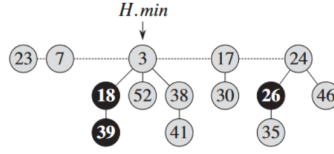
- (a) Starting from an empty binary heap, what is the resulting heap after successively inserting 7, 9, 5, 8, 11, 27, 4, 6, and 2? Please show your steps.
- (b) Following (a), what is the result after performing `removeMin()`? Please show your steps.

2. (16%) Binomial Heap

- (a) What is the smallest possible number of nodes in a binomial heap that contains 10 binomial trees?
- (b) Propose an algorithm to insert m nodes into a binomial heap of n elements in $O(m + \log n)$ worst-case time. Prove your bound.

3. (22%) Fibonacci Heap

- (a) Show the result after deleting the minimum in the Fibonacci heap in Figure 1. (show the step)
- (b) Show the result after decreasing 35 to 2 in the Fibonacci heap in Figure 1. (show the step)
- (c) Denote F_d as a smallest Fibonacci heap with degree d , Prove that the size of smallest Fibonacci heaps satisfy:
 $size(F_0) = 1$, $size(F_1) = 2$, and
 $size(F_d) = size(F_{d-1}) + size(F_{d-2})$, $d > 1$



4. (30%) Modified Fibonacci heap

Consider a “modified Fibonacci heap”, which is almost identical to the Fibonacci heap except that its decrease key operation is modified as follows (The piece of code that has been modified is indicated by the blue-colored text), It should be marked twice before cut:

Algorithm 1 Fib-Heap-Decrease-Key-Modified(H, x, k)

```

1: if  $k > key[x]$  then
2:   Error: “New key is greater than current key”
3: end if
4:  $key[x] \leftarrow k$ 
5:  $y \leftarrow p[x]$ 
6: if  $y \neq \text{NIL}$  and  $key[x] < key[y]$  then
7:   CUT-MODIFIED( $H, x, y$ )
8:   CASCADING-CUT-MODIFIED( $H, y$ )
9: end if
10: if  $key[x] < key[\min[H]]$  then
11:    $\min[H] \leftarrow x$ 
12: end if

```

Algorithm 2 Cut-Modified(H, x, y)

```

1: Remove  $x$  from the child list of  $y$ , decrement  $degree[y]$ 
2: Add  $x$  to the root list of  $H$ 
3:  $p[x] \leftarrow \text{NIL}$ 
4:  $mark[x] \leftarrow 0$ 

```

Algorithm 3 Cascading-Cut-Modified(H, y)

```

1:  $z \leftarrow p[y]$ 
2: if  $z \neq \text{NIL}$  then
3:   if  $mark[y] < 2$  then
4:      $mark[y] \leftarrow mark[y] + 1$ 
5:   else
6:     CUT( $H, y, z$ )
7:     CASCADING-CUT-MODIFIED( $H, z$ )
8:   end if
9: end if

```

- (a) (16 pts) Give a potential function and analyze the amortized cost of each of the following operations in a “modified Fibonacci heap” of size n . Note that the credits you receive also depends on how tight the bound you achieve (under the premise that the bound is true).
- i. (4 pts) Insertion
 - ii. (4 pts) Decrease-key
 - iii. (4 pts) Delete-min
 - iv. (4 pts) Union
- (b) (7 pts) What is the size of the smallest “modified Fibonacci heap” with degree 11? Justify your answers.
- (c) (7 pts) Let M_d be the size of the smallest “modified Fibonacci heap” with degree d . Please give a recursion formula for M_d in terms of $M_{d-1}, M_{d-2}, \dots, M_0$. Justify your answers.

5. (8%) **Disjoint Set**

Given the sets $\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}, \{9\}, \{10\}, \{11\}, \{12\}, \{13\}, \{14\}, \{15\}, \{16\}$, show the result of the following sequence of instructions: union(1,2), union(3,4), union(3,5), union(1,7), union(3,6), union(8,9), union(1,8), union(3,10), union(3,11), union(3,12), union(3,13), union(14,15), union(16,0), union(14,16), union(1,3), union(1,14) when the unions are

- (a) (4 pts) Weighted union by height without path compression.
- (b) (4 pts) Weighted union by rank with path compression.

6. (8%) **Disjoint Set Algorithm**

Suppose that we wish to add the operation $\text{Print-Set}(x)$, which is given a node x and prints all the members of the x 's set, in any order. Show how we can add just a single attribute to each node in a disjoint-set forest so that $\text{Print-Set}(x)$ takes a linear time in the number of members of x 's set and the asymptotic running times of the other operations are unchanged.

(Assume that we can print each member of the set in $O(1)$ time.)

Algorithm 4 *Disjoint-set forest*

```
1: function MAKE-SET( $x$ )
2:    $x.\text{parent} \leftarrow x$ 
3:    $x.\text{rank} \leftarrow 0$ 
4: end function
5: function UNION( $x, y$ )
6:    $x' \leftarrow \text{FIND-SET}(x)$ 
7:    $y' \leftarrow \text{FIND-SET}(y)$ 
8:   LINK( $x', y'$ )
9: end function
10: function LINK( $x, y$ )
11:   if  $x.\text{rank} > y.\text{rank}$  then
12:      $y.\text{parent} \leftarrow x$ 
13:   else
14:      $x.\text{parent} \leftarrow y$ 
15:     if  $x.\text{rank} == y.\text{rank}$  then
16:        $y.\text{rank} \leftarrow y.\text{rank} + 1$ 
17:     end if
18:   end if
19: end function
20: function FIND-SET( $x$ )
21:   if  $x \neq x.\text{parent}$  then
22:      $x.\text{parent} \leftarrow \text{FIND-SET}(x.\text{parent})$ 
23:   end if
24:   return  $x.\text{parent}$ 
25: end function
```
